IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 2 September 2022, accepted 26 September 2022, date of publication 5 October 2022, date of current version 11 October 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3212152

RESEARCH ARTICLE

Deepware: An Open-Source Toolkit for Developing and Evaluating Learning-Based and Model-Based Autonomous Driving Models

SHUNYA SEIYA[1], ALEXANDER CARBALLO[2,3], (Member, IEEE), EIJIRO TAKEUCHI[2,3], (Member, IEEE), AND KAZUYA TAKEDA[1,2,3], (Senior Member, IEEE)

[1]Graduate School of Informatics, Nagoya University, Furo-cho, Chikusa-ku, Nagoya, Aichi 464-8603, Japan
[2]Institute of Innovation for Future Society, Furo-cho, Chikusa-ku, Nagoya, Aichi 464-8603, Japan
[3]Tier IV Inc., Open Innovation Center, Nagoya University, Nakamura-ku, Nagoya, Aichi 450-6610, Japan

Corresponding author: Shunya Seiya (seiya.shunya@g.sp.m.is.nagoya-u.ac.jp)

This work was supported by the New Energy and Industrial Technology Development Organization (NEDO) under Grant JPNP18010.

ABSTRACT In recent decades, many learning-based autonomous driving systems have been proposed, and researchers have also created toolkits for developing these systems. These toolkits allow developers to train their models easily, and then test them using simulators. Existing toolkits for learning-based autonomous driving systems have some limitations however, which include inability to reuse modules or to perform accurate comparisons with model-based systems, as well as a lack of support for middle-to-middle models. As a solution, in this paper we introduce Deepware, an end-to-end toolkit for developing and evaluating learning-based autonomous driving models. Deepware includes the tools needed for collecting and evaluating datasets, training models, and evaluating models on simulators or in real-world environments using actual vehicles. Unlike existing toolkits, we used ROS as our platform, which is a set of software frameworks for robot software development widely used in autonomous driving systems as middleware, which allows cooperation with model-based systems. This approach also allows system modules to be shared when building models. In addition, it allows the comparison of learning-based and model-based methods under the same conditions. Moreover, by extracting features from model-based systems, our toolkit can also support middle-to-middle models. The proposed Deepware toolkit and dataset are available at: https://github.com/shunchan0677/deepware.

INDEX TERMS Supervised learning, predictive models, autonomous vehicles.

I. INTRODUCTION

In recent decades, autonomous driving and deep learning have become the focus of much research. As a result, various methods of learning-based autonomous driving have been proposed. These methods aim to achieve autonomous driving by creating models that can learn driving behavior from driving data through deep learning. Various methods of modeling human driving behavior in specific driving scenarios, such as lane following [1] and following a route that includes branches [2], have been proposed. Since learning-based methods build autonomous driving systems using driving data, if a large enough amount of data from a wide enough variety of driving situations can be learned, it may become possible to build autonomous driving systems that are more adaptable to real-world driving situations than conventional, model-based systems.

However, there are several problems with learning-based autonomous driving systems which still need to be resolved. Researchers have compared the performance of various learning-based methods, but it is not clear to what extent these methods can be used for real-world driving, compared to conventional model-based methods. In addition, the learning-based methods whose source code is publicly

The associate editor coordinating the review of this manuscript and approving it for publication was Jjun Cheng.

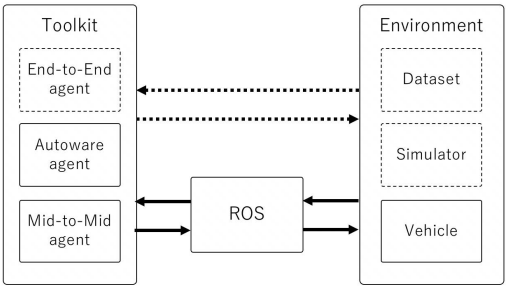

FIGURE 1. Overview of other learning-based toolkits and proposed toolkit. The operational features of existing toolkits are represented by the dotted lines in the figure. In addition to the operational features represented by the dotted lines, the proposed Deepware toolkit also includes the operations represented by the solid lines; a model-based agent (Autoware), a middle-to-middle based agent, ROS modules, and real vehicle support.

available use different evaluation criteria, training data, and driving environments, making it difficult to perform comprehensive comparisons in the same environment. In addition, most of the published source code is designed to be used in a simulator, and may directly use information on obstacles, self-position, and routes from the simulator. When migrating such systems into the environment of a real vehicle, a separate system for estimating the information needed in a real-vehicle environment is required.

In this paper, we build an integrated toolkit, which we call *Deepware*, to promote research and development in the field of learning-based autonomous driving systems. The proposed toolkit provides a framework in which learning-based autonomous driving models can be constructed using model-based autonomous driving frameworks such as ROS [3] and Autoware [4]. Differences between existing learning-based autonomous driving toolkits and the proposed toolkit are shown in the Figure1 and Table1. Existing learning-based toolkits are designed to run on a simulator and directly use information from the simulator, which results in additional development costs when these systems are deployed in real vehicles. In contrast, the proposed toolkit separates the simulator environment from the autonomous driving system using ROS, allowing easier migration to real vehicles, as well as the use of model-based components. This makes it possible to evaluate model-based methods and existing learning-based methods as baselines under the same conditions. Furthermore, middle-to-middle driving methods, which need to be integrated with a model-based system, can be implemented independently of the simulator and migrated into a real vehicle environment.

The contributions of the Deepware toolkit introduced in this paper are as follows:

- Increased reusability: By avoiding the tight coupling of autonomous driving systems with a simulator, our method allows the easy migration of automated driving models into real vehicles, by using the same implementation used during simulated driving.
- Comparison with model-based systems: The proposed toolkit makes it possible to compare the driving performance of model-based and learning-based methods within the same environment.
- Middle-to-middle model support: The proposed toolkit includes a method which allows the use of model-based autonomous driving resources in a learning-based autonomous driving toolkit, including both end-to-end and middle-to-middle agent methods.

This rest of this paper consists of the following sections: Section II describes learning-based autonomous driving approaches and existing learning-based toolkits. Section III describes the overall structure of our proposed model development method and each of the steps required when developing learning-based, autonomous driving models using the toolkit: dataset collection, model training, and model evaluation. Section IV describes the construction of five different driving models, and the experimental evaluation of these models in both simulator and real vehicle environments, using the proposed Deepware toolkit. Section V summarizes our findings and discusses future work.

II. RELATED WORK

Here we compare our toolkit with the other learning-based autonomous driving toolkits proposed in related work which are currently available. A comparison of these toolkits and the proposed method is shown in Table 1. The four toolkits compared are COiLTRAiNE [5], OATomobile [6], Transfuser [7], and our proposed Deepware toolkit.

COiLTRAiNE is open source software developed for tuning Conditional Imitation Learning models (CoIL [2]) using the CARLA simulator [8]. It operates within a Docker container [9] and allows users to collect training data, train the driving model, evaluate it on a data set, and evaluate the model on the simulator. Since this system only targets CoIL models, extension of the system to other models is not supported.

Another learning-based, autonomous driving toolkit called OATo Mobile [6], which allows users to build end-to-end driving models, has recently been proposed. It not only supports the CARLA simulator, but also OpenAI's Gym [10] for reinforcement learning. As baselines, implementations of RIP [6], DIM [11], and CoIL are also available, but pre-trained models are not provided.

Transfuser [7] is an end-to-end autonomous driving method which was developed for the CARLA Challenge Competition [12]. The Transfuser toolkit includes many pre-trained models, including their proposed method and other models proposed in related studies. The code needed to evaluate these models, using the evaluation metrics of the CARLA Challenge, is also provided. Moreover, when building their own models, users can fine-tune them using these pre-trained models.

As shown in Figure 2, there are three evaluation stages when developing learning-based autonomous driving systems; evaluation using an open dataset, evaluation using a

TABLE 1. Comparison of features of Deepware with those of other learning-based autonomous driving toolkits.

Toolkit	COilTRAiNE [5]	OATo mobile [6]	Transfuser [7]	Deepware
Simulator	CARLA [8]	CARLA / OpenAI-gym [10]	CARLA	CARLA
Evaluation metrics	Success rate	Success rate	CARLA Challenge	CARLA Challenge
End-to-End method support	CoIL [2]	CoIL [2]/DIM [11]/RIP [6]	CILRS [5]/LBC [15] / AIM [7]/ LateFusion [7]/ GeometricFusion [7]/ Transfuser [7]	IL [1]/Branch model [16]
Middle-to-Middle method support	×	×	×	✓
Pre-trained models	✓	×	✓	✓
ROS-support	×	×	×	✓
Model-based support	×	×	×	✓
Real vehicle support	×	×	×	✓

simulator, and evaluation using a real vehicle, and each of these evaluation stages involves three steps; dataset creation, model training, and model evaluation. Simulator stage and Real Vehicle stage modules can be shared when using Deepware, but when using a conventional system development process each of these nine steps must be implemented one by one.

Developers evaluate their systems in simulators and in real vehicles to test their models on-line. But in order to reduce development cost, the system should be able to share the same operating code during these two on-line evaluation steps. Related studies have only focused on evaluation using simulators however, so their systems will need to be rebuilt in order to perform evaluations using real vehicles. In addition, related studies have not compared the use of model-based and learning-based versions of their systems. The exclusive focus on the evaluation of learning-based systems in these studies makes it impossible to test learning-based systems under the same conditions as model-based systems. Moreover, related studies don't include methods based on middle-to-middle learning, because these methods require model-based modules, e.g., a perception module, a localization module, and so on.

To summarize, the problems with related studies are as follows:

- Additional development is required for evaluation using real vehicles.
- Unable to evaluate their learning-based systems under the same conditions as model-based systems.
- Unable to develop middle-to-middle driving models due to inability to share information with model-based systems.

In this study, when developing Deepware, we organized the steps needed to build a learning-based autonomous driving system, and proposed a ROS-based toolkit, which allows modules to be shared across each step of the development process. This approach also allows the same system modules to be used during both the simulator and real vehicle evaluation stages, reducing the cost of development. In addition, this makes it possible to evaluate model-based methods and existing learning-based methods under the same conditions.

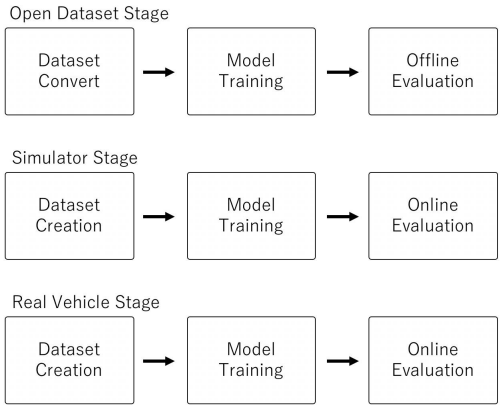

FIGURE 2. Process flow for learning-based autonomous driving when using the general toolkits. There are three evaluation stages when developing learning-based autonomous driving systems; evaluation using an open dataset, evaluation using a simulator, and evaluation using a real vehicle, and each of these evaluation stages involves three steps; dataset creation, model training, and model evaluation.

Furthermore, driving methods based on middle-to-middle learning, which require integration with a model-based system, can be implemented independently of the simulator, allowing them to be easily migrated into a real vehicle environment.

III. TOOLKIT DESIGN

A. OVERVIEW

In this section, we describe the overall structure of the Deepware toolkit and how to use it. Deepware was developed using TensorFlow and ROS. An illustration of the toolkit's overall structure is shown in Figure 3, and its use during the driving model development process is shown in Figure 4. As shown in Figure 3, Deepware can be used for Dataset Conversion, Model Training or Model Evaluation during the Open Dataset stage which is shown in orange line, Simulator stage which is shown in yellow line, or Real Vehicle stage which is shown in blue line. As shown in Figure 4, our toolkit divides the traditional nine-step development process, described in the previous subsection, into five development modules (A to E). Some of these modules are shared between the same steps and stages, reducing the cost of model development.

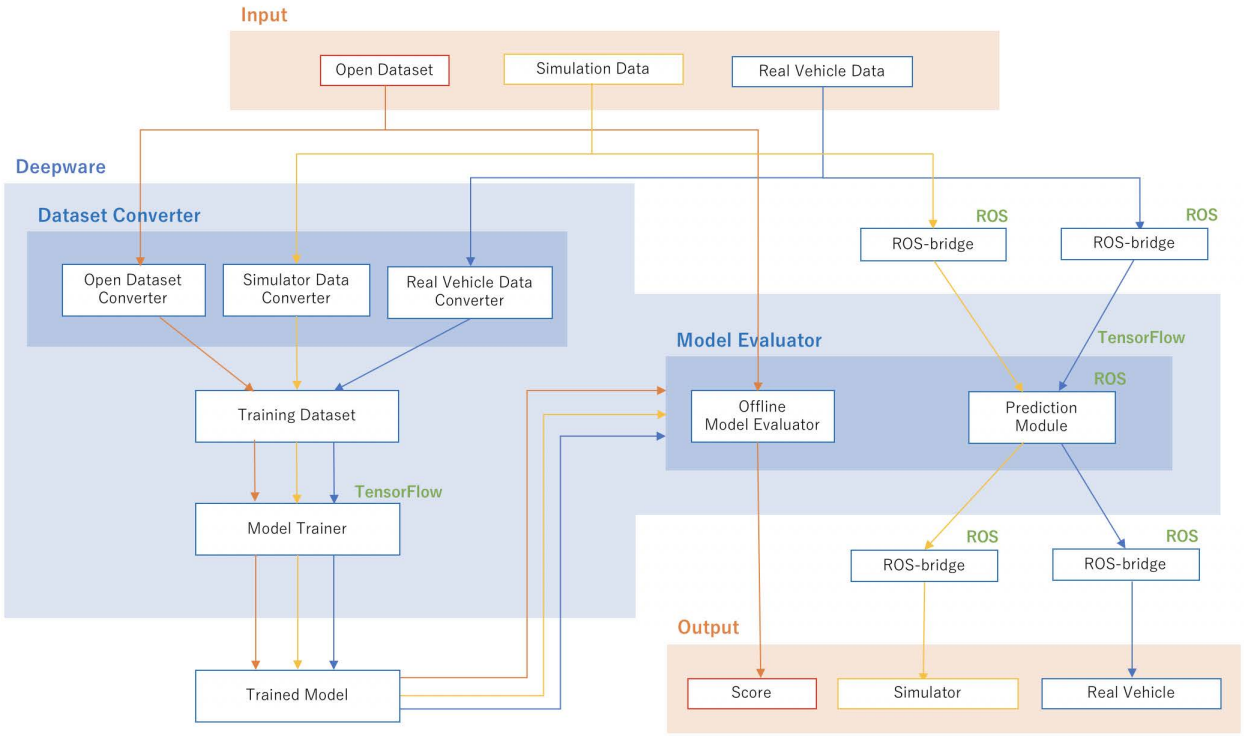

FIGURE 3. Overall structure of the Deepware toolkit. Our toolkit includes a dataset converter, model trainer and evaluator. ROS and TensorFlow are used. The development stage is selected by the user.

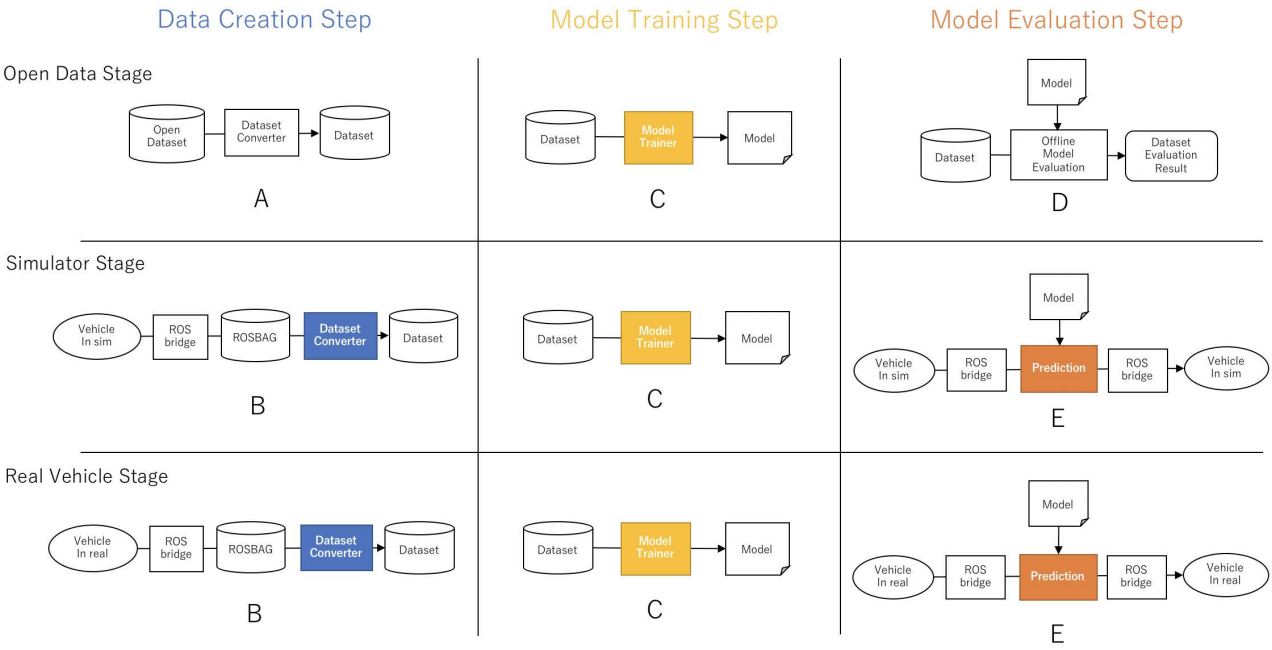

FIGURE 4. How to use the Deepware toolkit; Our toolkit divides the traditional nine-step development process into five development modules (A to E) for reducing the development cost.

As shown in Figure 4-A, Deepware users can import an open dataset to create a training dataset for the open dataset stage of development. As shown in Figure 4-B, users can also create a training dataset using their own rosbag dataset, collected by the user. Rosbag is the data format used by the Robot Operating System (ROS) for driving data. During the

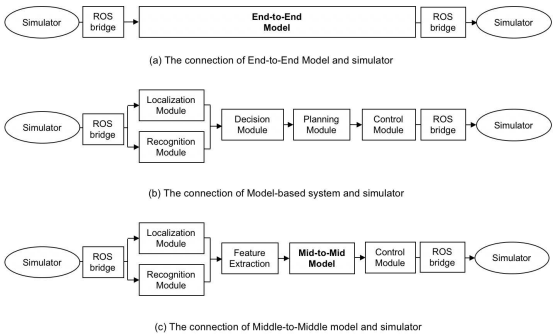

FIGURE 5. Connections between each type of control agent and the simulator. Users can choose from among three types of control agents for their prediction module: (a) an end-to-end driving method, (b) a model-based method, or (c) a middle-to-middle driving method.

simulator and real vehicle evaluation stages, the data converter module can be shared because a ROS bridge unifies the data formats. In this manner, our Deepware toolkit provides data converter features which can be used during the data creation step to create a training dataset from an open dataset or a rosbag dataset.

As shown in Figure 4-C, during the model training step the user can train an existing model using the training dataset built during the data creation step. The same model training module can be shared because the format of the training data is same for each stage.

As shown in Figure 4-D and E, during the model evaluation stage the user can evaluate their model using an open dataset, a simulator and a real vehicle. During the off-line evaluation step, shown in Figure 4-D, the offline evaluation feature are used to calculate the loss between the prediction results and the real-world data, using the dataset and the trained model. For on-line evaluation, shown in Figure 4-E, the model predicts the necessary driving signals based on sensor data transmitted through a ROS bridge. These signals can be used to control a vehicle in a simulator or in a real-world environment. As in the data creation step, ROS allows us to share modules and use a ROS bridge to unify the data formats.

During the development process, the modules used to create the dataset, to train the model, and to evaluate the model on-line can be shared, allowing users to lower their development costs by standardizing these modules.

During the model evaluation step, shown in Figure 4-E, Deepware users can choose from among three types of control agents for their prediction module: a model-based method, an end-to-end driving method, or a middle-to-middle driving method. Each type of control agent is illustrated in Figure 5. When the user chooses the end-to-end agent, as shown in 5(a), sensor information such as images are acquired from the simulator via a ROS bridge and input to the model. When using the model-based agent, as shown in Figure 5(b), the system calculates the necessary control signals based on acquired sensor information, using its recognition, decision, and control modules to control the vehicle in the simulator. In the case of the middle-to-middle driving agent, shown in Figure 5(c), the sensor information is passed through the recognition module of the model-based system to extract environmental features, which are then input to the model to calculate the control signals needed to navigate the vehicle in the simulator. This ease of switching between control agents allows developers to evaluate model-based and learning-based methods in the same environment.

In the following subsections we describe the design and features of our proposed Deepware toolkit. Subsection III-B describes the data collection and dataset creation functions. Subsection III-C describes the training of driving models. Subsection III-D describes the evaluation of these models, both on-line and off-line. Subsection III-E describes migration of the user's autonomous driving system to real vehicles. Finally, in subsection III-F, we discuss the driving models supported by the Deepware toolkit.

B. DATASET CREATION

As shown in Figure 4-A and B, the Deepware toolkit provides dataset creation features, which include a module for converting open datasets (Figure 4-A), a module for converting user datasets (Figure 4-B), and a data augmentation module.

Learning-based modeling methods require the collection of driving data and construction of training datasets. For example, in order to train an end-to-end model, it is necessary to create a dataset that includes images, as well as control signals or odometry. To train middle-to-middle models, we need point clouds, obstacle information, an HD map, waypoints, and signal recognition information, as shown in Figure 6. To prepare datasets which contain this information, users can either use existing, publicly available driving data, or they can collect new data using simulators or real-world driving environments. Unlike existing toolkits, Deepware unifies the data format by using ROS in both the simulator stage and the real vehicle stage, and the same learning modules are shared in each stage.

The Lyft Level 5 dataset [14] is one of the open datasets available in Deepware. This dataset was collected while driving through urban areas, and contains sensor information such as images and point clouds, as well as manually annotated object recognition information and an HD map. This data is converted into bird's eye view-like images and used as training data. An example is shown in the Figure 6, where the driving data is divided into a point cloud image, self-position information, an HD map, waypoints, etc., converted into a bird's eye view for input when training middle-to-middle methods.

The Deepware toolkit also supports users who want to collect a new set of driving data. The user can employ rosbag, a data collection function of ROS, to collect sensor and recognition data from various simulators that support ROS, or to collect driving data from real-world driving environments. The toolkit includes a tool that can extract each type of sensor data from the rosbag as training data, allowing users to create

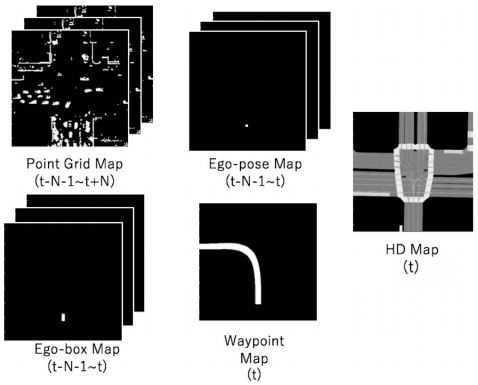

FIGURE 6. Examples of input features for middle-to-middle methods. Pre-processed data from sensor information must be created in advance, such as object recognition results.

a new dataset. In addition, the format of the ROS topics in the rosbag follows that of model-based system (Autoware), so that data collected while running Autoware can also be used for training. In addition, our toolkit includes a tool to automatically collect driving data while running Autoware in the CARLA simulator.

Many existing learning-based modeling methods realize lane following, for example, by collecting a limited amount of data and then using data augmentation to expand that data. In this toolkit, we provide two data augmentation tools: one for end-to-end methods, which uses a viewpoint transformation method based on camera geometry, and the other for middle-to-middle methods, which generates additional data using pseudo-movement of the ego vehicle's position. These augmentation methods can be used to improve model training, and thus driving performance.

C. MODEL TRAINING

As shown in Figure 4-C, the Deepware toolkit includes a model trainer module. The user has the option of training one of the existing models provided, the details of which are described in SectionIII-F, or they can train their own model. Training progress can be confirmed through observation of decreasing error values. By augmenting the test data used to measure the error value, we can confirm whether or not the error value is close to performance achieved in a real-world driving environment [17]. By managing the training code, trained models, and evaluation results using the simulators and datasets in this toolkit, users will be able to use, compare, and improve existing models in a manner that is reproducible, contributing to the advancement of research in the field of autonomous driving.

D. MODEL EVALUATION

As shown in Figure 4-D and E, the Deepware toolkit also includes model evaluation features. There are evaluation modules which use a dataset to perform off-line evaluation, as shown in Figure 4-D, and evaluation modules that work with a simulator or real vehicle, as shown in Figure 4-E. When evaluating with a dataset, Deepware uses Root Mean Square Error(RMSE) values, based on the observation that models with smaller RMSE values are able to predict driving signals more accurately. When evaluating models using the simulator, Deepware is able to connect to some types of autonomous driving simulators using ros-bridge. The simulator must be able to acquire sensor information from cameras, Lidar, etc., and be able to control the vehicle by sending control information, which are the types of information exchanged using ros-bridge. In this paper, we use the CARLA simulator.

TABLE 2. Infraction points for various accidents.

Infractions	Discounted points
Hitting static scenery	6
Hitting another vehicle	6
Hitting a pedestrian	9
Running a red light	3
Invading the opposite lane	2
Invading a sidewalk	2
Running a stop sign	2

CARLA Challenge metrics are used to evaluate model performance during simulated driving. The score for a model is calculated as shown in Equation 1, where N represents the number of routes, C is the percentage of driving that occurs along the specified route, and I represents infraction points. When using this evaluation equation, the longer the vehicle can follow the target path, the higher the model's score, while the model's score drops when accidents occur during driving. Table 2 shows the infraction points used for the CARLA Challenge evaluation. Deepware's use of the CARLA Challenge metrics for performance evaluation allows the comparison of learning-based and model-based autonomous driving methods.

$$Score(a) = \frac{1}{N}\sum_{i=1}^{N} \max(100C(a, r_i) - I(a, r_i), 0) \quad (1)$$

E. REAL ENVIRONMENT

By using ROS in Deepware, the toolkit can operate independently of the simulator, and information obtained using the simulator can be transferred to a real environment simply by connecting them, thus the cost of migration is low. When migrating a model from a simulator to a real-world environment without using this functionality, it is necessary to remove the module that obtained information from the simulator directly and reconfigure the model to function in a real environment. But migration can be performed easily using when using Deepware, thanks to ROS.

F. PROVIDED MODELS

There are four learning-based driving models supported by this toolkit: End-to-End, CoIL, Middle-to-Middle, and Point Grid-based. The End-to-End model is the method proposed by Bojarski et al [1]. and CoIL model is the method proposed

by Codevilla et al [2]. The Middle-to-Middle model is one that we developed [18], which is based on ChaufferNet [13]. The model takes a bird's-eye view feature image, rendered using Autoware's position estimation method, as an input, and outputs a waypoint several steps ahead. The Point Grid-based model [18] uses a point cloud image instead of the object recognition results for the feature image used by the middle-to-middle model. Since these two middle-to-middle approaches are an integration of learning-based and model-based methods, conventional implementation methods rely on self-position information from the simulator. But Deepware integrates the model-based method and implements the model independently from the simulator.

IV. EXPERIMENTAL EVALUATION

In this section, we describe three experiments we conducted to demonstrate that the Deepware toolkit can be used to evaluate the performance of a model, and that the model can then be used in a real-world environment. In the first experiment, we evaluate the prediction performance of the model using an open dataset. In the second experiment, we compare the driving performance of a model-based system and a learning-based system that includes a middle-to-middle model, using a simulator. In the third experiment, we demonstrate that the Deepware toolkit can be used to operate a vehicle in a real-world environment in the same manner as during the simulator evaluation stage.

A. OFF-LINE EVALUATION

In this section, we show that the prediction performance of each model can be evaluated using the Deepware toolkit and an open dataset of driving data. The configuration of the toolkit is shown in Figure 4-D, i.e., in the Open Data stage of the Model Evaluation step. We evaluated the model using the RMSE value when predicting the future position of the ego vehicle, using real driving data. We used Lyft's Level 5 dataset, which is one of the publicly available datasets included in Deepware. Since this dataset contains driving data collected from real automobiles, prediction accuracy in a real traffic environment can be measured. The dataset was divided into three parts for each driving scenario: training, evaluation, and testing.

In this evaluation, we compared four models: Bojarski et al.'s end-to-end method [1], CoIL [16], our ChaufferNet-based middle-to-middle model [18], and our point grid map model [18]. Normally, when using end-to-end and CoIL models, the control signals are estimated directly, but in order to match the conditions of the middle-to-middle model, instead we predicted the future position of the ego vehicle, (x, y). For our middle-to-middle and point grid map models, we used a Convolutional LSTM with the same configuration. Surrounding obstacle information was input as a historical object bounding box map when using the middle-to-middle model, while point cloud map information was input directly into the point grid map model. Traffic signal information is not included in the Level 5 dataset, so it was not considered in this evaluation.

TABLE 3. Conditions for Experiment 1 (off-line evaluation).

Feature map size	400 pix. × 400 pix.
Geometry size	80 m × 80 m
Current ego pose	320 pix. × 200 pix.
Hz of input data	2 Hz
# of frames of input data N	5 frames
# of input maps	32 maps
# of frames of output data N	5 frames
# of input maps	32 maps
Image configuration	binary image
Encoder architecture	3 × (2D-Conv + Pooling)
Regression architecture	3 × 2D-Conv + 3 × fully connected
LSTM architecture	2D-ConvLSTM
Optimizer	Adam
Learning rate	0.0001

TABLE 4. Results for Experiment 1 (off-line evaluation).

Methods	RMSE [m]
End-to-End [1]	2.09
Conditional Imitation Learning [16]	2.26
Middle-to-Middle [18]	0.35
Point grid based [18]	0.37

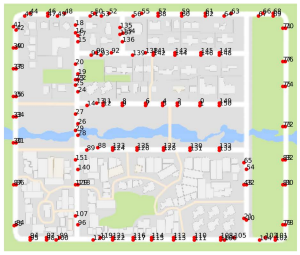

FIGURE 7. Example of course used for evaluation in the simulator environment.

The settings used during model training are shown in Table 3. In this experiment, we use the RMSE of the predicted future position of the ego vehicle as the evaluation index. We used 120 of the 180 driving sessions in the Level 5 dataset as training data, 30 were used as evaluation data, and the remaining 30 were used as test data.

By using the Deepware toolkit, we were able to evaluate these models using publicly available datasets. The experimental results for our off-line evaluation are shown in Table 4. These results show that the RMSE error values are less than 0.4 meters for the Middle-to-Middle and Point Grid-based models, while RMSE error for the End-to-End and CoIL models was greater than 2 meters. The reason for the higher performance of the middle-to-middle models is the use of an LSTM model for processing time series data.

B. ON-LINE EVALUATION

In this section, we show that the driving performance of model-based and learning-based methods can be compared in the same environment using the Deepware toolkit and the

TABLE 5. Results for Experiment 2 (on-line evaluation).

Methods	Points
Autoware [4]	82.3
End-to-End [1]	3.5
Conditional Imitation Learning [16]	2.6
Middle-to-Middle [18]	40.8
Point Grid Map based model [18]	43.4

CARLA simulator. The configuration of the toolkit is shown in Figure 4-E on simulator stage. In our evaluation using the off-line dataset, ego vehicle location prediction accuracy was compared, but in an off-line state the effect of feedback when controlling the vehicle cannot be considered. Therefore, in this experiment, CARLA Challenge metrics were used for performance evaluation. The surrounding vehicles were made to drive around randomly, while the ego vehicle needed to follow a specific target route while taking into consideration the movement of the other vehicles.

As experimental environments, we set up two separate experimental environments, one for training and the other for testing. The environment designated as Town01 in CARLA was used for training, and the environment designated as Town02 was used for testing, as shown in Figure 7. The five methods to be compared in this experiment are the model-based system (Autoware), the End-to-End model (Bojarski), CoIL, our ChaufferNet-based Middle-to-Middle model, and our Point Grid Map model. The data collection function of the toolkit was used to collect the training data, allowing Autoware to run automatically in CARLA and to collect the resulting driving data as a rosbag. Using Deepware's data collection function, we drove the car automatically in Town01 and collected waypoints, point clouds, images, driving signals, obstacle information, and the ego vehicle's position. During the test, each model was used to follow a target waypoint in Town02. The number of driving sessions during training was seven, while the number of driving sessions during testing was five. The evaluation results for each model are the average CARLA Challenge scores of the five test driving sessions.

As a result, we were able to evaluate different types of models using the CARLA simulator and the Deepware toolkit. The scores for each model are shown in Table 5. Our results show that Autoware, a model-based system, achieved better performance than the four learning-based methods. The reason for this is that the Middle-to-Middle model cannot follow the targeted waypoints in the middle of driving scenarios. When comparing the learning-based methods, the Middle-to-Middle model performed better than the End-to-End model, as was also the case in the off-line evaluation results.

Figure 8 shows the output of the Middle-to-Middle model during an evaluation driving session. We can see that the model is able to learn how to return to the route leading to the waypoint, even when the waypoint is shifted, because data augmentation was used during training.

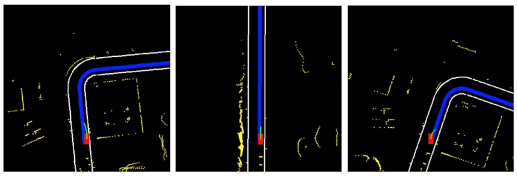

FIGURE 8. Example of prediction results when using the Middle-to-Middle model. The red rectangle shows the ego vehicle's position and the blue line shows the motion prediction results. The examples on the left and right of the figure show that the vehicle is being prevented from deviating from its lane.

C. REAL-WORLD ENVIRONMENTS

In this section, we demonstrate that the Deepware toolkit can be used to operate a vehicle in a real-world environment in the same manner as during our evaluation in the simulator stage. The configuration of the toolkit is shown in Figure 4-E, i.e., in the Real Vehicle stage of the Model Evaluation step. The mobile robot used as the vehicle is shown in Figure 9. The mobile robot is controlled using speed and curvature control signals, like a real car, so it cannot turn on the spot, for example. The robot and its sensors are connected to a PC, which receives the sensor data and sends the control signals. The robot is equipped with a motor driver that replicates the speed signal input. Using the CoIL model, which is one of the models used in the experiments described in the section IV-B, we conducted an experiment that involved having a mobile robot follow a route along a pathway with multiple branches, which is described in detail in one of our previous papers [16]. Unlike the other toolkits, it was easy for Deepware to collect forward images from the robot's camera, joystick operation information, and odometry from the mobile robot's camera and joystick drivers, since the tool used to operate the mobile robot was compatible with ROS. In this experiment, we show that dataset creation, model training, and driving in a real-world environment can be performed in the same manner as in the simulator environment, described in the section IV-B.

The first step is to create training data. The operator manually navigated the vehicle using a joystick, and the vehicle collected point cloud, camera image, joystick operation, and odometry data, and stored it as a rosbag. The rosbag data was then expanded using Deepware's data augmentation tool, in the same manner as in the previous experiment, and it was then used as training data. In order to train the CoIL model to follow the correct branches along the robot's route, a target direction vector was calculated using point cloud data from the Lidar scanner and position estimation results from Autoware.

Next, the model was trained using the training data. During this step, we measured the model's error against the augmented test data, so that we could create a model with high driving performance. The details of the experimental conditions are shown in Table 6. In this experiment, Autoware estimated the robot's position and calculated the target

TABLE 6. Experimental conditions for Experiment 3 (real world navigation).

training data	52417 × 9
validation data	20000 × 9
CNN input	Frontal image(200 × 66 × 3)+ target direction vector
CNN output	Curvature
CNN structure	Norm layer + convolutional layer 5 layers + fully-connected layer 4 layers
activation	Leaky-Relu
training times	20 epochs
mini-batch	10
learning-rate	0.00001
constant λ	0.05
velocity of robot	0.5 m/s
target point for training period	Self-position for 160 frames
target point for test period	Self-position for 3 m

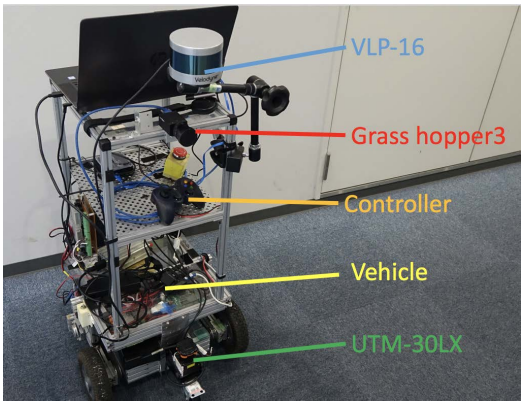

FIGURE 9. Mobile robots that used in Experiment 3.

direction vector, while the output calculated by the model was used to control the vehicle.

The results of the test sessions are shown in Figure 10, demonstrating that the model was able to navigate branches encountered along the route. Through these experiments, we have demonstrated the advantages of using ROS when creating learning-based, autonomous driving models with the Deepware toolkit, and how the same procedure can be used when building models to control vehicles in either the simulator or real vehicle stage of development. In this experiment, we used a mobile robot and the simple, real-world environments instead of a car and real traffic environments. Although there are differences between these two environments in terms of speed, surrounding obstacles, traffic rules, etc., the process flow (using sensor data as inputs, processing this data using a driving model, and controlling the vehicle using speed and angular velocity control signals) is consistent with the use of a real vehicle. Therefore, this experiment demonstrates how Deepware can be used in a real-world environment for learning-based automated driving.

D. DISCUSSION

The results of our off-line evaluation experiment, shown in Table 4, demonstrate how models can be evaluated with the

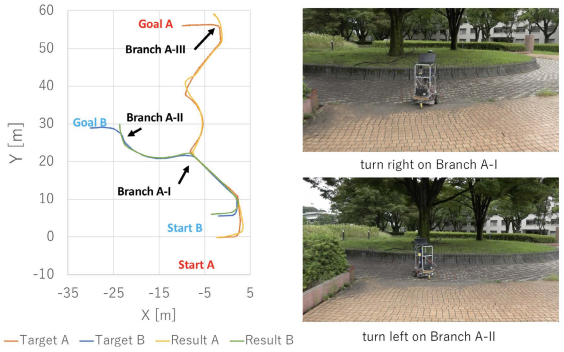

FIGURE 10. Left: Trajectory of the mobile robot when traveling along a branching route. Right: Example of evaluation route.

Deepware toolkit by using RMSE values. In this experiment, the Lyft dataset was converted into a user dataset using Deepware's data creation feature, as shown in Figure 4-A, and then used for model training (Figure 4-C) and model evaluation (Figure 4-D). By building the system this way, the training module shown in Figure 4-C can be reused in the other development stages.

The results of our on-line evaluation experiment, shown in Table 5, demonstrate that models can be evaluated using the CARLA Challenge metrics as a functionality of the Deepware toolkit, and that model-based and learning-based systems can be compared in same environment. Our results show that the model-based method (Autoware) achieved better performance than all four of the learning-based methods. It is also possible, by comparing the performance of these learning-based and model-based methods when operating under the same conditions, to identify possible reasons for the differences in their scores.

The results of our real-world evaluation experiment, shown in Figure 10, demonstrate that the CoIL model can be evaluated using the Deepware toolkit and a real robot vehicle, and that the same procedure can be used when building a model to control vehicles in either the simulator or real vehicle stages.

In order to improve the performance of learning-based models, it will be necessary to create a model that is more generalized, so that it can perform well in a variety of driving environments. In addition, since the learning-based models were often unable to reach waypoints, a method that imposes more constraints in such scenarios will be necessary. When comparing the learning-based methods, the middle-to-middle model outperformed the end-to-end model during the on-line evaluation, a trend that was also observed during the off-line evaluation.

V. CONCLUSION

In this paper, we have introduced the Deepware toolkit for the development of learning-based, autonomous driving models. Deepware is based on the ROS platform, and is designed to work with Autoware, a model-based autonomous driving toolkit. It includes tools for data creation, model training,

and model evaluation. Since we developed our toolkit using ROS, it is easy for Deepware to support middle-to-middle autonomous driving methods, which are not supported in other toolkits, and to migrate models into real-world environments for evaluation.

Our goals for the future include supporting all of the models supported by other toolkits, and the introduction of sim2real and real2sim models, so that models trained on simulator can be used in real-world environments, and vice versa. In addition to supporting a wider variety of models, we also hope to be able to acquire more standardized driving models, trained using large-scale datasets, and provide them to our users.

REFERENCES

[1] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Müller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to end learning for self-driving cars," 2016, *arXiv:1604.07316*.

[2] F. Codevilla, M. Müller, A. Lopez, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 4693–4700.

[3] *Documentation—ROS Wiki*. Accessed: Oct. 4, 2022. [Online]. Available: http://wiki.ros.org/

[4] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, "An open approach to autonomous vehicles," *IEEE Micro*, vol. 35, no. 6, pp. 60–68, Nov./Dec. 2015.

[5] F. Codevilla, E. Santana, A. Lopez, and A. Gaidon, "Exploring the limitations of behavior cloning for autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9329–9338.

[6] F. Angelos, T. Panagiotis, M. Rowan, R. Nicholas, L. Sergey, and G. Yarin, "Can autonomous vehicles identify, recover from, and adapt to distribution shifts?" in *Proc. IEEE Int. Conf. Mach. Learn.*, Nov. 2020, pp. 3145–3153.

[7] A. Prakash, K. Chitta, and A. Geiger, "Multi-modal fusion transformer for end-to-end autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 7077–7087.

[8] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn.*, 2017, pp. 1–16.

[9] *Home—Docker*. Accessed: Oct. 4, 2022. [Online]. Available: https://www.docker.com/

[10] *openai/gym*. Accessed: Oct. 4, 2022. [Online]. Available: https://github.com/openai/gym

[11] N. Rhinehart, R. McAllister, and S. Levine, "Deep imitative models for flexible inference, planning, and control," in *Proc. IEEE Int. Conf. Learn. Represent.*, 2020, pp. 1–19.

[12] *Results of CARLA AD Challenge 2019*, Accessed: May 6, 2020. [Online]. Available: https://carlachallenge.org/results-challenge-2019/

[13] M. Bansal, A. Krizhevsky, and A. Ogale, "ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst," 2018, *arXiv:1812.03079*.

[14] R. Kesten, M. Usman, J. Houston, T. Pandya, K. Nadhamuni, A. Ferreira, M. Yuan, B. Low, A. Jain, P. Ondruska, S. Omari, S. Shah, A. Kulkarni, A. Kazakova, C. Tao, L. Platinsky, W. Jiang, and V. Shet. (2019). *Lyft Level 5 AV Dataset 2019*. [Online]. Available: https://level5.lyft.com/dataset/

[15] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Proc. IEEE Int. Conf. Robot Learn.*, 2019, pp. 1–10.

[16] S. Seiya, A. Carballo, E. Takeuchi, C. Miyajima, and K. Takeda, "End-to-end navigation with branch turning support using convolutional neural network," in *Proc. IEEE Int. Conf. Robot. Biomimetics (ROBIO)*, Dec. 2018, pp. 499–506.

[17] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, "On offline evaluation of vision-based driving models," 2018, *arXiv:1809.04843*.

[18] S. Seiya, A. Carballo, E. Takeuchi, and K. Takeda, "Point grid map-based mid-to-mid driving without object detection," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Oct. 2020, pp. 2044–2051.

SHUNYA SEIYA received the B.S. degree in engineering and the M.S. degree in informatics from Nagoya University, in 2017 and 2019, respectively, where he is currently pursuing the Ph.D. degree with the Graduate School of Informatics.

His research interests include deep learning and autonomous driving.

ALEXANDER CARBALLO (Member, IEEE) received the Dr. (Eng.) degree from the Intelligent Robot Laboratory, University of Tsukuba, Japan.

From 1996 to 2006, he worked as a Lecturer at the School of Computer Engineering, Costa Rica Institute of Technology. From 2011 to 2017, he worked in research and development at Hokuyo Automatic Company Ltd. Since 2017, he has been a Designated Assistant Professor at the Institutes of Innovation for Future Society, Nagoya University, Japan. His research interests include LiDAR sensors, robotic perception, and autonomous driving.

EIJIRO TAKEUCHI (Member, IEEE) received the bachelor's, master's and Ph.D. degrees from the Intelligent Robot Laboratory, University of Tsukuba, Japan.

From 2008 to 2014, he worked at Tohoku University, Japan, as an Assistant Professor. From 2014 to 2016, he held the title of a Designated Professor at Nagoya University, Japan, where he has been continued working at the Graduate School of Informatics as an Associate Professor, since 2016. His research interests include localization, map-ping in robotics, and autonomous driving systems.

KAZUYA TAKEDA (Senior Member, IEEE) received the B.E.E., M.E.E., and Ph.D. degrees from Nagoya University, Nagoya, Japan.

Since 1985, he has been with the Advanced Telecommunication Research Laboratories and the KDD Research and Development Laboratories, Japan. In 1995, he started a research group for signal processing applications at Nagoya University. He is currently a Professor with the Institutes of Innovation for Future Society, Nagoya University. His research interests include investigating driving behavior using data centric approaches and utilizing signal corpora of real driving behavior. He is a member of the Board of Governors of the IEEE ITS Society.